

Formation FPGA

TP5 : Domaines d'horloge

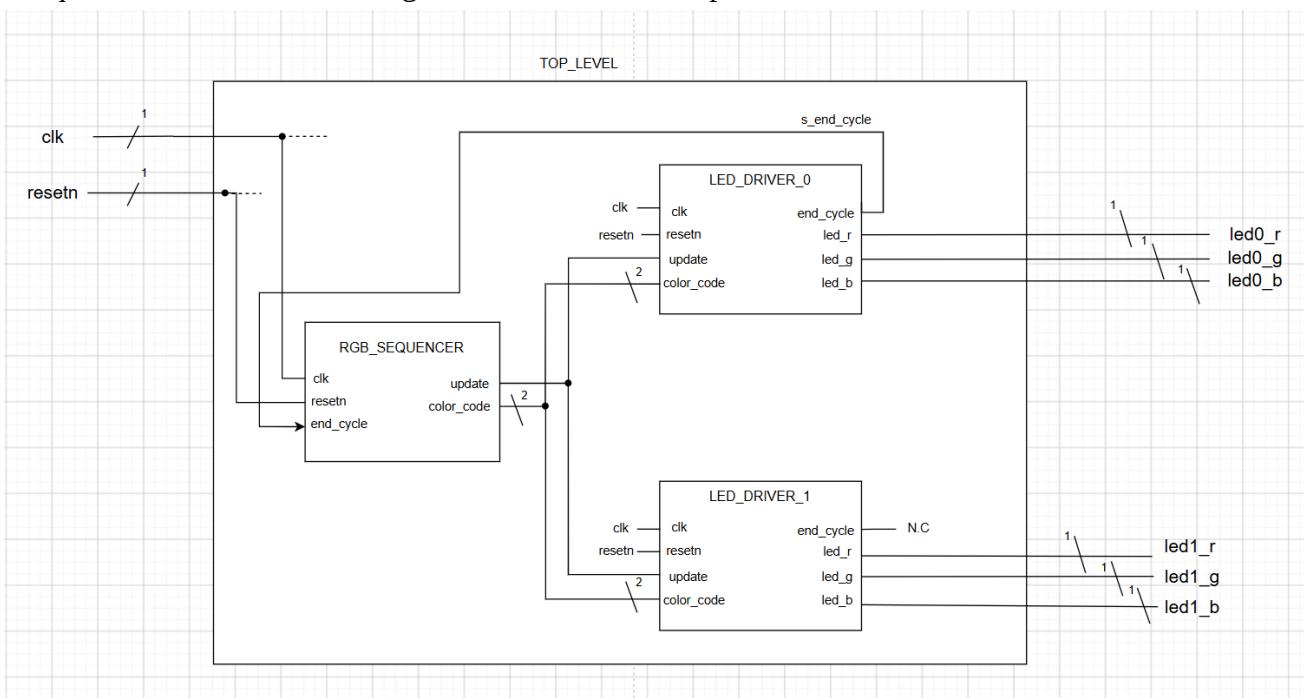
1. Introduction

L'objectif de ce cinquième TP est la modélisation et la réalisation d'un système multi-horloges sous Vivado via des modules de synchronisation CDC. La manipulation consiste en sa description matérielle (séquenceur RGB + bloc CDC), suivie de sa validation par analyseur logique interne (ILA) et de l'analyse des signaux à l'oscilloscope. Enfin, l'implémentation et la programmation du Bitstream sur la carte Cora Z7 permettront la démonstration physique de la transmission du signal de contrôle entre deux domaines d'horloge.

2. PARTIE I

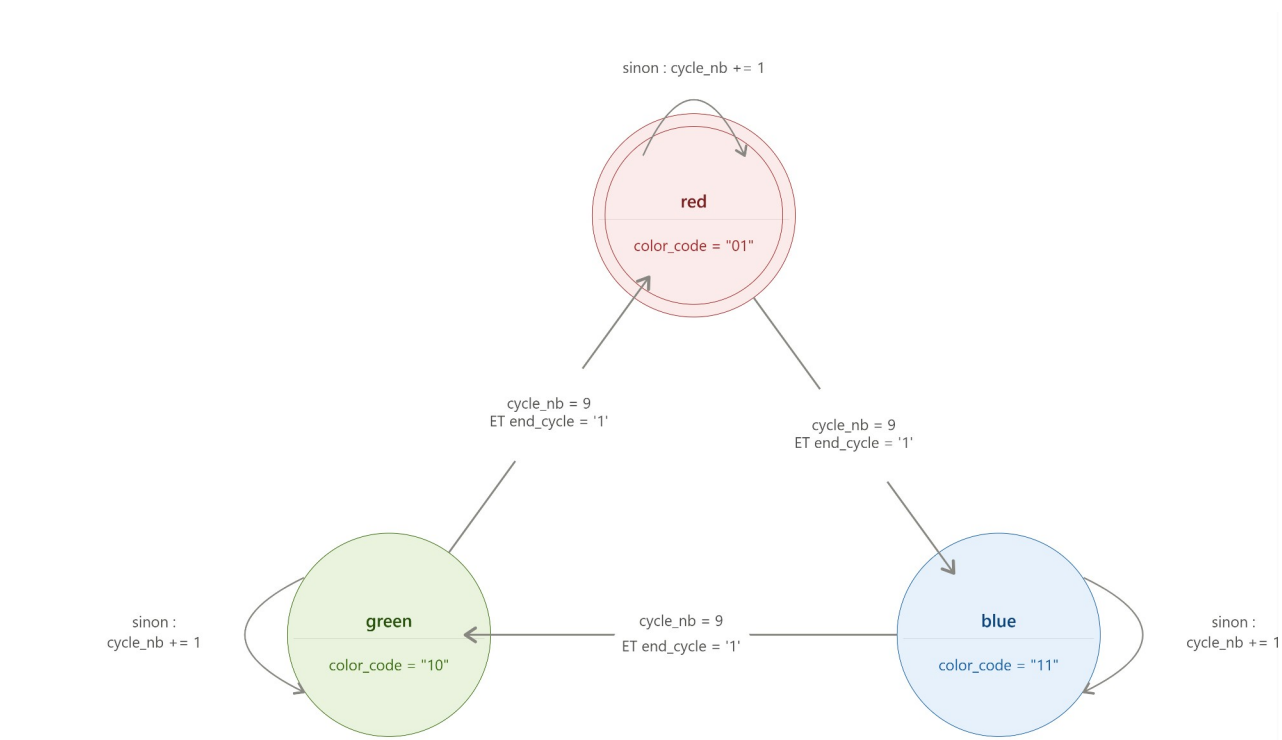
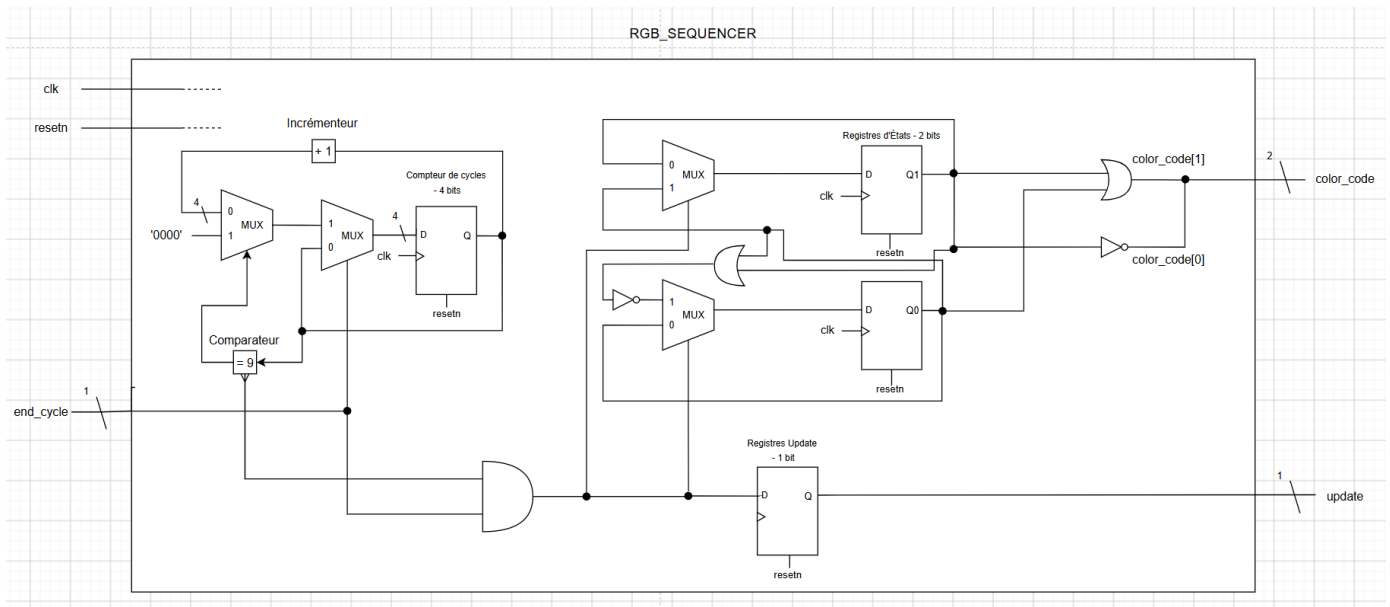
2.1. Question 1

Pour cette première partie de notre application, le module **LED_Driver** développé précédemment sera de nouveau implémenté avec quelques légères modifications d'adaptation à la nouvelle architecture RTL. Afin de respecter les spécifications données, l'établissement d'une autre machine d'états devient alors une solution concrète. Le module **LED_Driver** doit recevoir les codes couleur dans un certain ordre limité par un nombre de cycles **OFF/ON** défini pour chaque LED. Cette réflexion amène donc à la nécessité d'implémenter un séquenceur avec une machine de 3 états (**Rouge - Bleu - Vert**) et un compteur interne pour le nombre de cycles. Les deux modules LED Driver étant sur le même fonctionnement pour cette première mise en place, un seul séquenceur et une seule horloge suffisent dans ce cas précis.



Le Top Level représente ainsi nos trois blocs principaux implémentés et reliés pour les prochaines applications. Pour cette première représentation, l'horloge est identique, par conséquent les signaux d'entrée et de sortie des modules **LED_Driver** sont envoyés et reçus sur les mêmes fronts d'horloge.

Remarque : L'architecture peut s'apparenter à une configuration Maître-Esclave. La sortie 'end_cycle' du module **LED_Driver_1** est non connectée étant donné que deux sorties ne peuvent pas se relier à une entrée d'un autre module. Avec une horloge identique, la sortie 'end_cycle' du **LED_Driver_0** est amplement suffisante pour le bon fonctionnement du système global.



L'architecture matérielle du module **RGB_SEQUENCER** repose sur deux blocs principaux : la logique de temporisation et le cœur de la machine à états de Moore :

- Le bloc de temporisation s'appuie sur un registre de 4 bits (Compteur de cycles) associé à un incrémenteur et à une cascade de multiplexeurs. À chaque coup d'horloge, si le signal d'activation 'end_cycle' est actif, le compteur s'incrémente de 1 jusqu'à atteindre la valeur 9 pour obtenir le nombre de cycles **ON/OFF** exact, puis il est remis à zéro. Un comparateur valide cette fin de cycle et la combine avec 'end_cycle' via une porte **AND** pour produire le signal de transition.
- Le cœur séquentiel de la FSM utilise deux bascules D (Registre d'États) pour mémoriser l'état présent. La logique d'état futur, pilotée par les **MUX** et le signal en sortie de la porte **AND**, force la séquence d'états **00 → 01 → 10 → 00**.
- En sortie, un décodeur combinatoire composé d'une porte **OR** avec les équations ('color_code[1]' = Q1 **OR** Q0) et d'un inverseur **NOT** ('color_code[0]' = **NOT** Q1) traduit l'état courant en signal couleur.
- Enfin, le signal 'Enable' (en sortie de la porte **AND**) passe à travers une bascule D (Registre Update) pour délivrer la sortie d'actualisation 'update' avec une seule impulsion de cycle d'horloge.

La logique combinatoire implémentée pour la machine d'états ainsi que le décodeur de sortie peuvent également être visualisées avec ces tables de vérité :

<u>Commande MUX 'Enable'</u>	<u>Q1</u>	<u>Q0</u>	<u>État présent</u>	<u>D1</u>	<u>D0</u>	<u>État futur</u>
0	0	0	Rouge	0	0	Rouge
0	0	1	Bleu	0	1	Bleu
0	1	0	Vert	1	0	Vert
1	0	0	Rouge	0	1	Bleu
1	0	1	Vert	1	0	Vert
1	1	0	Bleu	0	0	Rouge
1	1	1	Non utilisé			

<u>Q1</u>	<u>Q0</u>	<u>Couleur</u>	<u>'color_code[1]'</u>	<u>'color_code[0]'</u>
0	0	Rouge	0	1
0	1	Bleu	1	1
1	0	Vert	1	0
1	1	Non utilisé		

Ces tables de vérité permettent ainsi d'obtenir les équations logiques nécessaires :

FSM : D1 = Q0 ; D0 = NOT (Q1 OR Q0)

Décodeur de sortie : color_code[1] = Q1 OR Q0 ; color_code[0] = NOT

2.2. Question 2

RGB_SEQUENCER.VHD

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity rgb_sequencer is
    port (
        clk          : in std_logic;
        resetn        : in std_logic;
        end_cycle     : in std_logic;
        update        : out std_logic;
        color_code    : out std_logic_vector(1 downto 0)
    );
end rgb_sequencer;

architecture Behavioral of rgb_sequencer is
    type state_type is (red, green, blue);
    signal current_state : state_type;
    signal cycle_nb      : integer range 0 to 9;

begin
    -- FSM en 1 seul process synchrone (sorties enregistrées)
    process(clk, resetn)
    begin
        if resetn = '0' then
            current_state <= red;
            cycle_nb      <= 0;
            update        <= '0';
            color_code    <= "01"; -- Rouge par défaut
        elsif rising_edge(clk) then
            -- Par défaut, l'impulsion update dure 1 seul cycle
            update <= '0';

            if end_cycle = '1' then
                if cycle_nb = 9 then
                    cycle_nb <= 0;
                    update    <= '1'; -- Impulsion du signal de mise à jour
                end if;

                -- Changement d'état et mise à jour de la couleur
                case current_state is
                    when red => --Par défaut sur Rouge - 10 cycles après -> Bleu
                        current_state <= blue;
                        color_code    <= "11";
                    when blue => -- 10 cycles après -> Vert
                        current_state <= green;
                        color_code    <= "10";
                    when green => --Retour de la boucle -> Rouge
                        current_state <= red;
                        color_code    <= "01";
                    when others =>
                        current_state <= red;
                        color_code    <= "01";
                    end case;
                else
                    cycle_nb <= cycle_nb + 1;
                end if;
            end if;
        end if;
    end process;
end Behavioral;
```

Le code VHDL ci-dessus retranscrit notre schéma RTL théorique du module RGB_Sequencer. Les transitions de la machine d'états sont gérés directement avec le signal interne « current_state » différant de l'approche combinatoire avec le signal 'next_state' afin d'avoir un design de FSM à un seul process synchrone.

Chaque fin de cycle ON/OFF permet de vérifier si le nombre de cycles nécessaire a été atteint pour assurer la transition vers l'état suivant. Si tel est le cas, une simple impulsion sur un cycle d'horloge du signal de sortie 'update' est effectuée pour la mise à jour de la gestion des LEDs dans les modules **LED_Driver**.

Remarque : Contrairement à l'architecture classique (registre d'état séparé de la logique de transition combinatoire et des sorties combinatoires), le module **RGB_Sequencer** regroupe l'ensemble de sa logique : comptage, transition d'état et génération des sorties, dans un unique process synchrone, avec des sorties directement enregistrées ('update', 'color_code'). Ce choix se justifie par la nature du signal 'update' : devant être une impulsion strictement calibrée à un seul coup d'horloge, sa génération directement depuis une bascule (plutôt que par une expression combinatoire dérivée de plusieurs conditions) élimine tout risque d'aléa de commutation (glitch) qui pourrait apparaître transitoirement sur un chemin combinatoire avant stabilisation.

Cette approche permet également d'anticiper la prochaine application mettant en œuvre plusieurs horloges pour chaque module de gestion de LED.

TOP LEVEL.VHD

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity top_level is
  generic (
    TOGGLE_DELAY : integer := 99999999 -- Valeur par défaut pour la synthèse FPGA
  );
  port (
    clk      : in  std_logic;
    resetn   : in  std_logic;
    -- Sorties RGB LED0
    led0_r   : out std_logic;
    led0_g   : out std_logic;
    led0_b   : out std_logic;
    -- Sorties RGB LED1
    led1_r   : out std_logic;
    led1_g   : out std_logic;
    led1_b   : out std_logic
  );
end top_level;

architecture Behavioral of top_level is
  -- Déclaration du module RGB_Sequencer
  component rgb_sequencer
    port (
      clk      : in  std_logic;
      resetn   : in  std_logic;
      end_cycle : in  std_logic;
      update    : out std_logic;
      color_code : out std_logic_vector(1 downto 0)
    );
  end component;
```

```

-- Déclaration du module LED Driver
component led_driver
  generic (
    TOGGLE_DELAY : integer := 99999999
  );
  port (
    clk      : in  std_logic;
    resetn   : in  std_logic;
    update    : in  std_logic;
    color_code : in  std_logic_vector(1 downto 0);
    end_cycle : out std_logic;
    led_r     : out std_logic;
    led_g     : out std_logic;
    led_b     : out std_logic
  );
end component;

-- Signaux d'interconnexion internes entre instances
signal s_update    : std_logic;
signal s_color_code : std_logic_vector(1 downto 0);
signal s_end_cycle0 : std_logic;

begin

  -- Instance 1 Séquenceur
  Inst_RGB_Sequencer : rgb_sequencer
  port map (
    clk      => clk,
    resetn    => resetn,
    end_cycle => s_end_cycle0,
    update    => s_update,
    color_code => s_color_code
  );

  -- Instance LED_Driver 0 -Maitre
  Inst_LED_Driver_0 : led_driver
  generic map (
    TOGGLE_DELAY => TOGGLE_DELAY
  )
  port map (
    clk      => clk,
    resetn    => resetn,
    update    => s_update,
    color_code => s_color_code,
    end_cycle => s_end_cycle0,
    led_r     => led0_r,
    led_g     => led0_g,
    led_b     => led0_b
  );

  -- Instance LED_Driver 1 -Esclave
  Inst_LED_Driver_1 : led_driver
  generic map (
    TOGGLE_DELAY => TOGGLE_DELAY
  )
  port map (
    clk      => clk,
    resetn    => resetn,
    update    => s_update,
    color_code => s_color_code,
    end_cycle => open,
    led_r     => led1_r,
    led_g     => led1_g,
    led_b     => led1_b
  );

end Behavioral;

```

Le **Top Level** représente ainsi notre architecture globale avec les différentes instances déclarées et interconnectées entre elles selon le schéma RTL théorique .

2.3. Question 3

```
-- Verifie a tout instant que LED1 reproduit exactement LED0 |
checking : process(s_clk)
begin
    if rising_edge(s_clk) then
        assert (s_led0_r = s_led1_r and s_led0_g = s_led1_g and s_led0_b = s_led1_b)
            report "- ERREUR - LED1 ne reproduit pas LED0"
            severity error;
        end if;
    end process;

simulation : process
begin
    -----
    -- Initialisation : Reset et demarrage sur Rouge
    -----
    s_resetrn <= '0';
    wait for period * 4;
    s_resetrn <= '1';

    wait until (s_led0_r = '1' or s_led0_g = '1' or s_led0_b = '1');
    assert (s_led0_r = '1' and s_led0_g = '0' and s_led0_b = '0')
        report "- ERREUR - La sequence ne démarre pas sur Rouge"
        severity error;
    report "- INFO - OK : sequence démarree sur Rouge" severity note;

    -----
    -- Apres 10 clignotements Rouge (COLOR_DURATION), doit passer au Vert
    -- Attente de la duree calculee, puis resynchronisation sur
    -- l'evenement reel (LED qui s'allume) plutot que de faire
    -- confiance a un delai fixe : robuste quelle que soit la duree
    -- exacte d'une phase "LED_ON"
    -----
    wait for COLOR_DURATION - period;
    wait until (s_led0_r = '1' or s_led0_g = '1' or s_led0_b = '1');
    assert (s_led0_g = '0' and s_led0_r = '1' and s_led0_b = '0')
        report "- ERREUR - Transition Rouge -> Vert incorrecte apres 10 cycles"
        severity error;
    report "- INFO - OK : transition vers Bleu apres 10 cycles Rouge" severity note;

    -----
    -- Apres 10 clignotements Bleu, la FSM doit passer au Bleu
    -----
    wait for COLOR_DURATION - period;
    wait until (s_led0_r = '1' or s_led0_g = '1' or s_led0_b = '1');
    assert (s_led0_b = '0' and s_led0_r = '0' and s_led0_g = '1')
        report "- ERREUR - Transition Bleu -> Vert incorrecte apres 10 cycles"
        severity error;
    report "- INFO - OK : transition vers Vert apres 10 cycles Bleu" severity note;

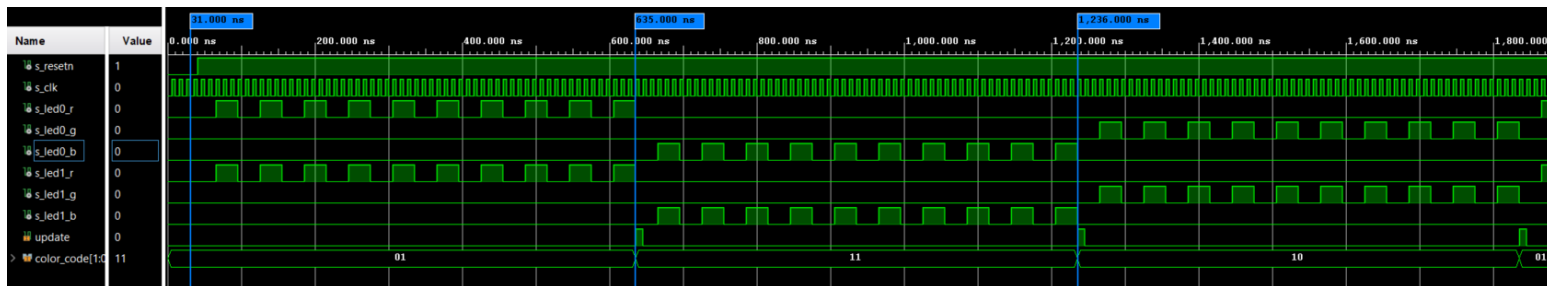
    -----
    -- Apres 10 clignotements Bleu, la FSM reboucle sur l'état Rouge
    -----
    wait for COLOR_DURATION - period;
    wait until (s_led0_r = '1' or s_led0_g = '1' or s_led0_b = '1');
    assert (s_led0_r = '1' and s_led0_g = '0' and s_led0_b = '0')
        report "- ERREUR - La boucle ne revient pas sur Rouge apres 10 cycles Bleu"
        severity error;
    report "- INFO - OK : boucle complete R -> V -> B -> R validee" severity note;

    -----
    -- Fin de simulation
    -----
    report "- SIMULATION TERMINEE AVEC SUCCES -" severity note;
    wait;
end process;

end behavioral;
```

L'objectif du testbench ci-dessus sera de vérifier la FSM du module RGB_Sequencer et son impact sur les modules de gestion LED Driver :

- Dans un premier temps, des conditions sont posées afin de vérifier que le passage à la bonne couleur de LED est effectuée.
- Les dix cycles ON/OFF doivent être également respectées.
- Le module **LED_Driver_1** doit reproduire selon la cohérence de l'application, les mêmes actions et aux mêmes cycles d'horloge que le module **LED_Driver_0**.



Les différents signaux de sortie de couleurs de chaque LED sont bien réalisés dans l'ordre suivant Rouge → Bleu → Vert. Le signal 'update' apporte une impulsion sur un cycle d'horloge et permet aux modules **LED_Driver** le changement de couleurs. Le signal de sortie 'color_code' transmet la couleur décodée selon les spécifications. La simulation est validée.

2.4. Question 4

Afin de gérer deux signaux d'horloge différents, le Top Level doit prendre en entrée deux signaux d'horloge appelés clkA et clkB. Il suffit ensuite de modifier les entrées des instances correspondantes.

- 'clkA' pour les modules **RGB_Sequencer** et **LED_Driver_0** (Ces modules gèrent aussi la cadence des changements de couleurs du module **LED_Driver_1**)
- 'clkB' pour le module **LED_Driver_1**

```
entity top_level is
    generic (
        TOGGLE_DELAY : integer := 99999999
    );
    port (
        clkA : in std_logic;
        clkB : in std_logic;
        resetsn : in std_logic;
        -- Sorties RGB LED0
        led0_r : out std_logic;
        led0_g : out std_logic;
        led0_b : out std_logic;
        -- Sorties RGB LED1
        led1_r : out std_logic;
        led1_g : out std_logic;
        led1_b : out std_logic
    );
end top_level;
```

```
Instance_RGB_Sequencer : rgb_sequencer
    port map (
        clk => clkA,
        resetsn => resetsn,

        Instance_LED_Driver_0 : led_driver
            generic map (
                TOGGLE_DELAY => TOGGLE_DELAY
            )
            port map (
                Instance_LED_Driver_1 : led_driver
                    generic map (
                        TOGGLE_DELAY => TOGGLE_DELAY
                    )
                    port map (
                        clk => clkB,
                        resetsn => resetsn,
```

2.5. Question 5

Quelques modifications sont effectuées dans le testbench pour répondre à l'ajout d'une autre horloge :

```
signal s_resetrn : std_logic := '0';
signal s_clkA : std_logic := '0'; -- Horloge sequenceur + LED0 : 250 MHz
signal s_clkB : std_logic := '0'; -- Horloge LED1 : 50 MHz
signal s_led0_r, s_led0_g, s_led0_b : std_logic;
signal s_led1_r, s_led1_g, s_led1_b : std_logic;

-- clkA : 250 MHz -> periode 4 ns
constant hpA : time := 2 ns;
constant periodA : time := 2*hpA;

-- clkB : 50 MHz -> periode 20 ns
constant hpB : time := 10 ns;
constant periodB : time := 2*hpB;

constant TB_TOGGLE_DELAY : integer := 2;
constant NB_CYCLE : integer := 10;

-- Duree d'une couleur complete, calculees sur l'horloge du sequenceur (clkA)
constant COLOR_DURATION : time := periodA * NB_CYCLE * 2 * TB_TOGGLE_DELAY;

-- Marge de securite pour LED1
-- une latence de quelques cycles clkB avant que le changement ne soit
-- visible cote LED1, en plus du delai normal de propagation.
constant MARGIN : time := periodB * 4;

horlogeA : process
begin
    wait for hpA;
    s_clkA <= not s_clkA;
end process;

horlogeB : process
begin
    wait for hpB;
    s_clkB <= not s_clkB;
end process;
```

2.6. Question 6



La simulation permet de constater un dysfonctionnement flagrant : le signal représentant la **LED_1** en sortie reste figée sur la couleur Rouge alors que plusieurs impulsions du signal 'update' ont eu lieu au cours du temps de simulation. Cela s'explique simplement par le fait que le module **LED_Driver_1** n'a pas pu saisir sur front montant ce signal 'update' avec une horloge beaucoup

plus lente que le cadenceur comme représenté sur la deuxième image. Par conséquent, la mise à jour de l'entrée 'color_code' ne peut pas avoir lieu. Des solutions doivent donc palier ce problème.

2.7. Question 7

Afin de régler le conflit de domaine d'horloge rencontré précédemment, une première méthode consiste à modifier la largeur temporelle de l'impulsion directement à la source, ou aussi appelée **Pulse Stretching**. L'impulsion 'update' de 4 ns produite par 'clkA' est rallongée à l'aide d'un compteur interne pour que sa durée dépasse la période de l'horloge de destination 'clkB' (20 ns). Le signal élargi est maintenu à l'état haut pendant au minimum 6 cycles de 'clkA' (soit 24 ns), ou idéalement 10 cycles (40 ns) pour garantir une marge de sécurité de 2 cycles complets de 'clkB', ce qui garantit son intersection avec au moins un front montant de 'clkB'.

Au moment du franchissement de domaine d'horloge, le signal traverse une chaîne de deux bascules de synchronisation (**2 Flip-Flop**). Ce double étagement est indispensable pour enlever tout risque de **métastabilité** causé par une violation des temps d'établissement setup ou de maintien hold à l'entrée du domaine d'horloge 'clkB'.

Une fois le signal stabilisé et sécurisé contre la métastabilité, il reste présent pendant plusieurs cycles de 'clkB'. Pour éviter que le module **LED_Driver_1** n'interprète ce niveau haut prolongé comme plusieurs commandes de mise à jour successives, un détecteur de front montant isole uniquement le premier cycle de la transition.

Cette technique permet de délivrer l'impulsion d'un cycle attendue, mais elle présente le désavantage de dépendre du ratio entre les deux fréquences, imposant de recalculer la durée d'étirement en cas de modification des horloges.

2.8. Question 8

```
begin

-----
-- Etirement d'impulsion (clkA = 250 MHz)
-- Étire le pulse de 4 ns à 40 ns (10 cycles clkA = 2 cycles clkB)
-----
process(clkA, resetn)
begin
    if resetn = '0' then
        stretching_cpt <= 0;
        s_update_stretched <= '0';
    elsif rising_edge(clkA) then
        if s_update = '1' then
            stretching_cpt <= 9; -- 1 cycle initial + 9 de décompte = 10 cycles (40 ns)
            s_update_stretched <= '1';
        elsif stretching_cpt > 0 then
            stretching_cpt <= stretching_cpt - 1;
            s_update_stretched <= '1';
        else
            s_update_stretched <= '0';
        end if;
    end if;
end process;
```

```

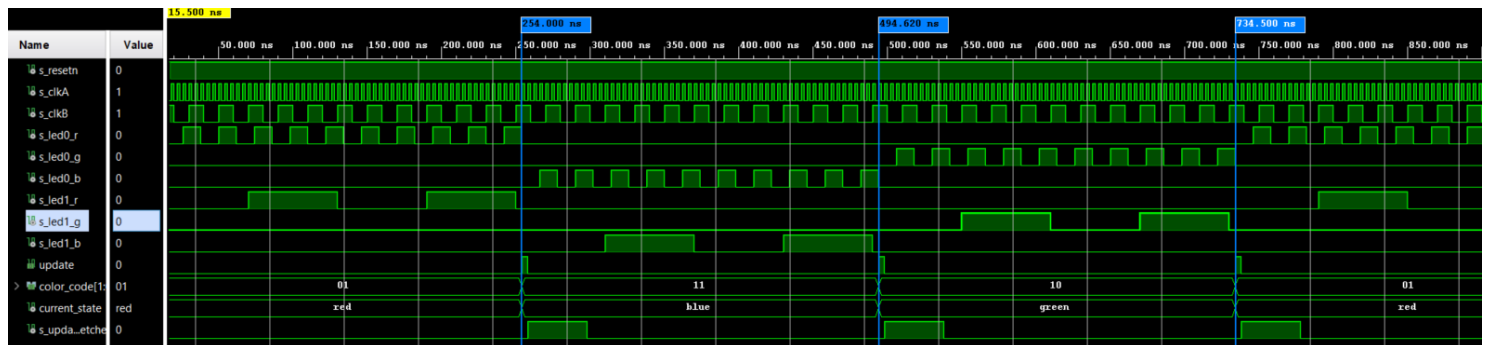
-----
-- Synchronisation 2-Flip Flop (Metastabilite) + Detecteur de front (clkB - 50 MHz)
-----
process(clkB, resetn)
begin
    if resetn = '0' then
        r1_stage <= '0';
        r2_stage <= '0';
        r3_stage <= '0';
    elsif rising_edge(clkB) then
        r1_stage <= s_update_stretched; -- Absorption de la metastabilite
        r2_stage <= r1_stage;          -- Signal propre etire
        r3_stage <= r2_stage;          -- Registre d'un cycle de retard
    end if;
end process;

-- Generation de l'impulsion de 1 cycle clkB pour le module LED_Driver_1
s_update_clkB <= r2_stage and not r3_stage;

```

Des modifications sont apportées dans le **Top Level** pour directement assurer la méthode du Pulse Stretching. Les modules **LED_Driver** ou du **séquenceur** n'étant pas propices à ces modifications, ils sont juste utilisés de manière générique en envoyant les signaux correspondants.

Pour le domaine d'horloge '*clkA*', un décompteur est nécessaire afin de pouvoir générer le signal '*update*' étiré sur plusieurs cycles en sortie du séquenceur comme expliqué précédemment (10 cycles pour une durée de 40 ns). Ce signal '*update*' « étiré » est donc envoyé vers la suite de bascules : Des signaux intermédiaires sont déclarés représentant chaque étape de l'enchaînement des bascules Flip Flop afin de gérer la métastabilité et finalement générer une seule impulsion sur un cycle d'horloge pour le module **LED_Driver_1**.



Le signal '*update_stretched*' permet dans cette nouvelle configuration le changement de couleur pour le module **LED_Driver_1**.

Le séquenceur cadence bien les cycles de clignotement en fonction du module **LED_Driver_0** :

- 10 cycles ON/OFF pour une horloge '*clkA*' de 250 MHz
- 2 cycles ON/OFF pour une horloge '*clkB*' de 50 MHz donc 5 fois moins rapide !

La simulation avec la solution du **Pulse Stretching** est donc validée.

Remarque : Une autre solution plus optimale consisterait à faire basculer le signal ‘*update*’ à chaque impulsion pour un niveau logique inverse (un « *toggle* ») afin d'assurer un état permanent, facilement enregistré par les bascules. Ce changement de niveau logique est ensuite reconverti en impulsion côté récepteur, ce qui rend cette méthode plus robuste et indépendante d'un quelconque changement de fréquence d'horloge.

2.9. Question 9

Clocking Wizard (6.0)

Documentation IP Location Switch to Defaults

Component Name: clk_wiz_0

Clocking Options Output Clocks Port Renaming MMCM Settings Summary

☐ Enable Clock Monitoring

Primitive

☒ MMCM ☐ PLL

Clocking Features

☒ Frequency Synthesis ☐ Minimize Power

☒ Phase Alignment ☐ Spread Spectrum

☐ Dynamic Reconfig ☐ Dynamic Phase Shift

☐ Safe Clock Startup

Jitter Optimization

☒ Balanced ☐ Minimize Output Jitter

☐ Maximize Input Jitter filtering

Dynamic Reconfig Interface Options

☒ AXI4Lite ☐ DRP ☐ Phase Duty Cycle Config ☐ Write DRP registers

Input Clock Information

	Input Clock	Port Name	Input Frequency(MHz)		Jitter Options	Input Jitter	Source
☒	Primary	clk_in1	125.000	10.000 - 800.000	UI	0.010	Single e
☐	Secondary	clk_in2	100.000	75.000 - 150.000		0.010	Single e

Output Clocks

The phase is calculated relative to the active input clock.

Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)
		Requested	Actual	Requested	Actual	Requested
☒ clk_out1	clk_out1	250.000	250.000000	0.000	0.000	50.000
☒ clk_out2	clk_out2	50.000	50.000000	0.000	0.000	50.000
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000

☐ USE CLOCK SEQUENCING

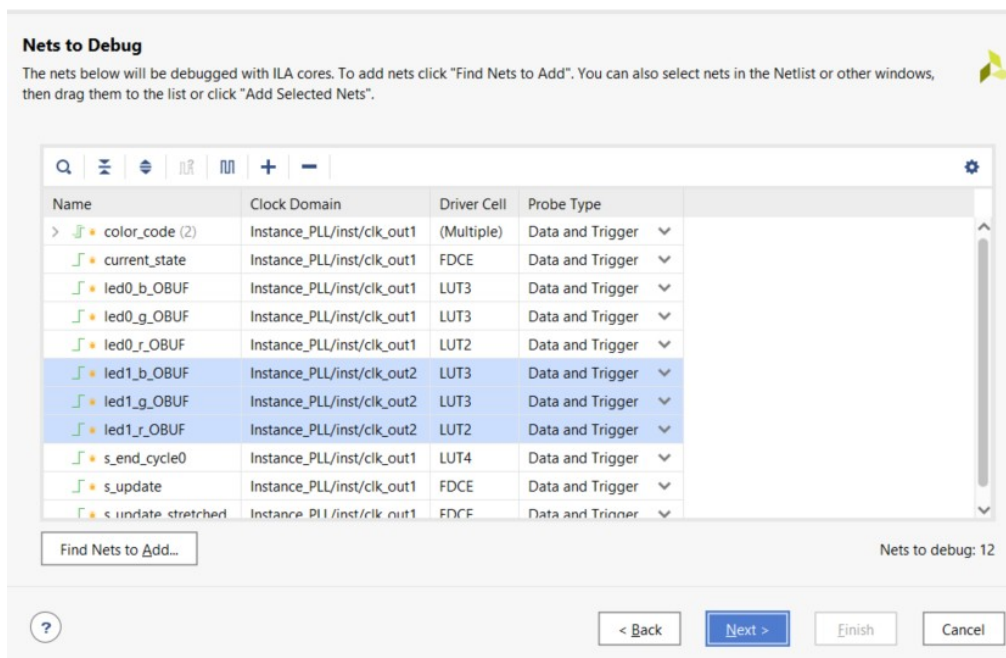
Clocking Feedback

Output Clock	Sequence Number	Source	Signaling
clk_out1	1	☒ Automatic Control On-Chip	☒ Single-ended
clk_out2	1	☐ Automatic Control Off-Chip	☐ Differential
clk_out3	1	☐ User-Controlled On-Chip	
clk_out4	1	☐ User-Controlled Off-Chip	
clk_out5	1		

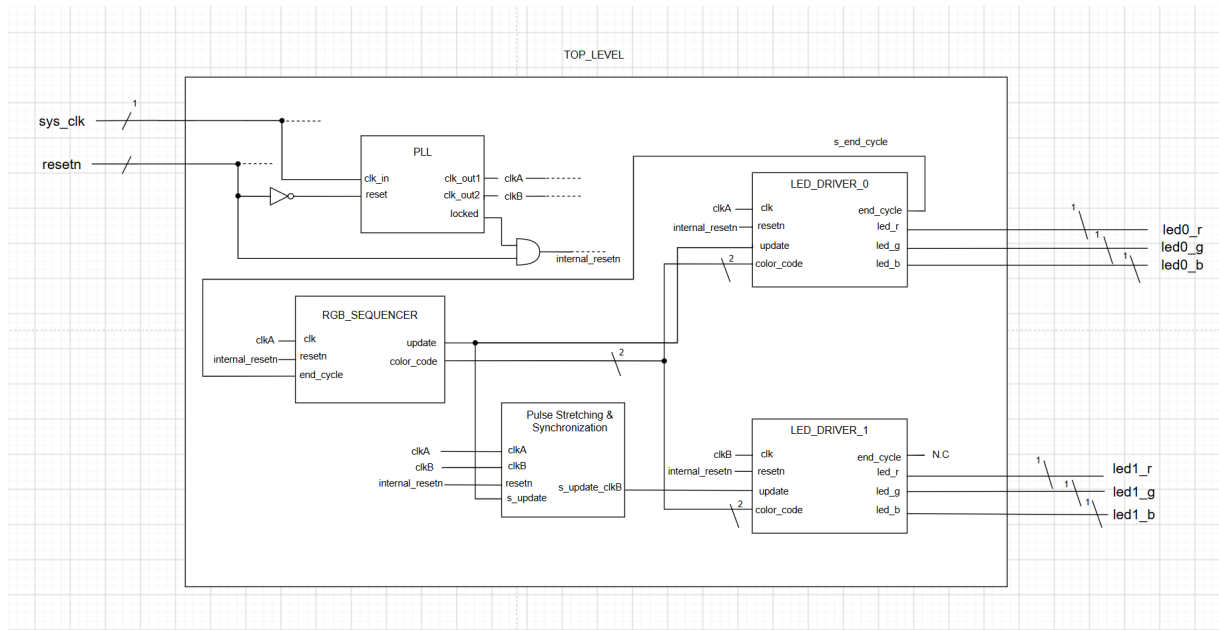
La section **Clocking Wizard** listé dans l'IP Catalog fournit le module PLL nécessaire pour fournir les deux horloges différentes à partir de celle donnée en entrée.

La déclaration exacte de la fréquence d'entrée 125 MHz au lieu de 100 MHz dans l'IP **Clocking Wizard** est indispensable pour que la **PLL** calcule correctement ses coefficients de multiplication et de division. Si la valeur 100 MHz était conservée alors que l'oscillateur matériel de la Cora Z7 délivre 125MHz, les horloges générées seraient 25% plus rapides que prévu (312,5 MHz et 62,5 MHz au lieu de 250 MHz et 50 MHz). Cette incohérence provoquerait des violations de temps de propagation et risquerait d'empêcher le verrouillage de la **PLL** (*'locked'* = '0'), bloquant ainsi l'ensemble du système sous *'reset'*. De plus, les rapports de timing seraient aussi falsifiés démontrant un résultat différent que ceux obtenus sur carte en temps réel.

2.10. Question 10



La section **Set Up Debug** va permettre de visualiser lors de la programmation sur carte du fichier bitstream, les différents changements d'états, triggers de signaux internes ou des entrées et sorties de l'architecture globale.



La version finale du **Top Level** prend en compte les nouvelles modifications ainsi que l'ajout du **PLL**. Les différentes horloges en sortie sont donc envoyées vers les blocs correspondants ('clkA' pour le séquenceur et le module **LED_Driver_0**, 'clkB' pour le module **LED_Driver_1**)

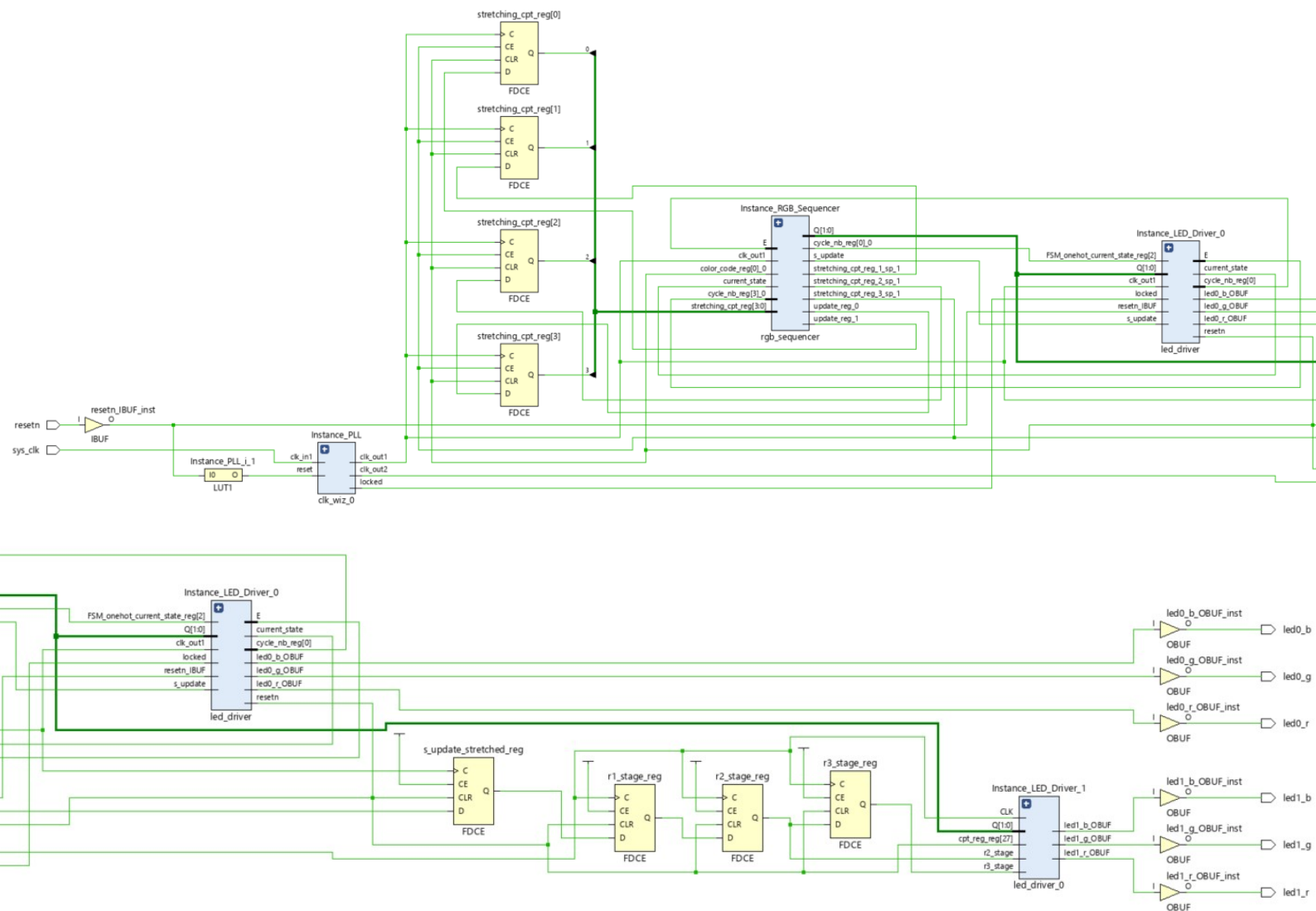
2.11. Question 11

Estimation théorique des ressources matérielles

L'estimation théorique repose sur le décompte fonctionnel des éléments décrits l'architecture globale avant l'étape de synthèse :

- Pour les registres et bascules (73 FFs estimées) : Les besoins en mémoire synchrone sont dominés par les deux pilotes de LED (led_driver), nécessitant chacun un compteur de 27 bits pour atteindre la valeur de TOGGLE_DELAY, soit 54 bascules au total, associées à 6 bascules pour les registres de sortie RGB. Le module rgb_sequencer requiert 3 bascules (2 pour le codage des états et 1 pour l'impulsion update). Le bloc de changement de domaine d'horloge (CDC) utilise 4 bascules pour le décompte d'étirement stretching_cpt, 1 bascule pour le signal étiré s_update_stretched, et 3 bascules pour la chaîne de synchronisation anti-métastabilité (r1_stage, r2_stage, r3_stage). Enfin, 2 bascules sont allouées au réseau de sécurité du signal locked de la PLL.
- Pour la logique combinatoire et les tables de vérité (environ 78 LUTs estimées) : La majeure partie de la logique combinatoire dérive des deux compteurs de 27 bits (logique d'incréméntation et comparateurs d'égalité avec TOGGLE_DELAY), représentant environ 66 LUTs. Le reste des ressources combinatoires regroupe la logique de transition de la machine à états du séquenceur (environ 3 LUTs), le contrôle de décréméntation du compteur d'étirement (environ 4 LUTs), le détecteur de front ascendant du CDC (1 LUT) ainsi que les portes logiques d'inversion du reset système.

- Pour l'arithmétique (14 CARRY4 estimées) : Dans l'architecture Zynq-7000, la réalisation d'un additionneur/compteur s'appuie sur les chaînes CARRY4, chaque bloc traitant 4 bits. Les deux compteurs de 27 bits nécessitent ainsi exactement 7 primitives CARRY4 chacun, soit un total de 14 blocs.
- Pour les entrées/sorties et ressources d'horloge (8 IOB, 3 BUFG, 1 PLL) : L'interface physique utilise 8 broches : 1 entrée d'horloge maître (sys_clk), 1 entrée de réinitialisation (resetn) et 6 sorties pour les commandes des LED RGB (led0_r,g,b et led1_r,g,b). La gestion des domaines d'horloge nécessite 1 bloc matériel PLL au sein de l'IP clk_wiz_0 et 3 tampons d'horloge globaux (BUFG) pour distribuer sys_clk, clkA (250 MHz) et clkB (50 MHz).



Ref Name	Used	Functional Category
FDCE	76	Flop & Latch
LUT4	71	LUT
CARRY4	14	CarryLogic
LUT3	8	LUT
LUT5	7	LUT
OBUF	6	IO
LUT2	6	LUT
LUT1	4	LUT
FDPE	4	Flop & Latch
LUT6	2	LUT
IBUF	1	IO

Résultats post-synthèse et analyse des écarts

- Les résultats de synthèse indiquent 80 bascules au total. Les 76 bascules FDCE correspondent aux registres réinitialisés à l'état bas (niveau logique '0'), couvrant les compteurs (54 FFs), les sorties RGB, la machine à états et les étages de resynchronisation CDC. Les 4 bascules FDPE sont des primitives à préchargement haut (niveau logique '1') lors de la réinitialisation, issues de la logique interne de l'IP `clk_wiz_0` et de l'arbre de synchronisation du signal '`locked`'. Ce mécanisme garantit le maintien du circuit sous reset tant que la PLL n'est pas verrouillée.
- L'outil de synthèse a favorisé les structures à 4 entrées pour deux raisons directes : la sévérité de la contrainte temporelle du domaine '`clkA`' à 250 MHz ($T = 4$ ns), où les LUT4 présentent un temps de traversée inférieur à celui d'une LUT6 complète, et la faible complexité des équations logiques du circuit (détection de front à 2 entrées, inversion de reset à 1 entrée, compteur d'étirement à 4 bits).
- L'augmentation observée à 98 LUTs découle des optimisations physiques exécutées par Vivado. Afin de limiter le facteur de forme sur des signaux de contrôle critiques ('`internal_resetn`', '`s_update`') qui dégraderaient le timing à **250 MHz**, l'outil duplique localement certaines portes combinatoires.

Remarque : Si un signal doit être envoyé à 30 endroits différents en même temps, le fil devient long et le signal met du temps à voyager. À 250 MHz, le signal n'a que 4 nanosecondes pour arriver. Pour éviter qu'il ne soit en retard, Vivado préfère dupliquer la même porte LUT en plusieurs exemplaires et les placer juste à côté des composants qui en ont besoin. Cela consomme plus de LUTs, mais le circuit devient plus rapide.

Traverser une grosse LUT à 6 entrées prend un tout petit peu plus de temps que traverser une petite LUT à 3 ou 4 entrées. Comme le domaine d'horloge à 250 MHz est très exigeant, Vivado découpe les calculs en plusieurs petites LUT4 alignées. Une fonction logique qui aurait pu tenir dans 1 seule grande LUT se retrouve découpée et étalée sur 2 ou 3 petites LUTs plus rapides.

2.12. Question 12

Le rapport d'implémentation final montre un respect des contraintes de timing :

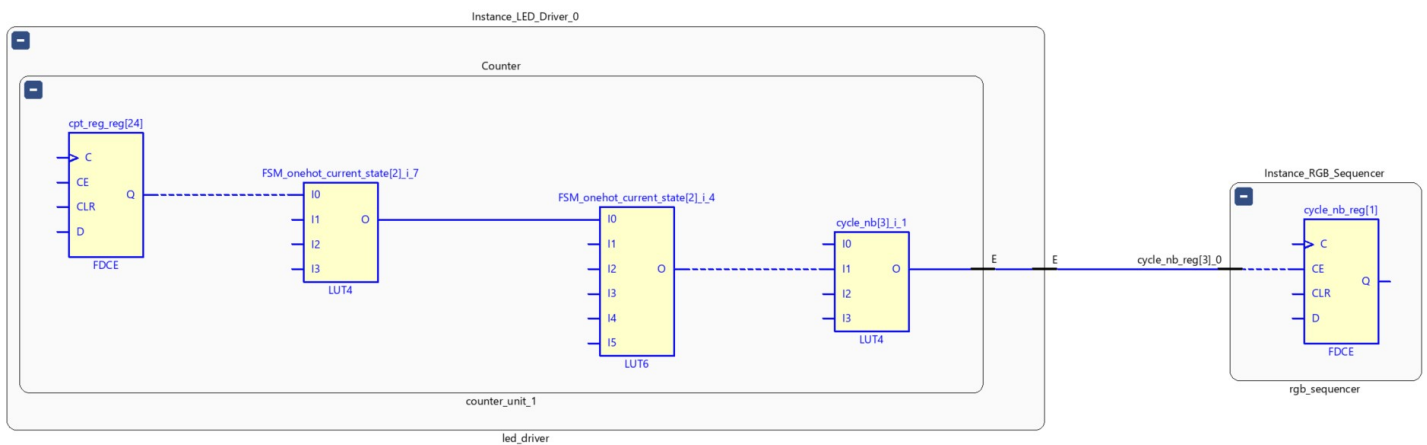
Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0,371 ns	Worst Hold Slack (WHS): 0,139 ns	Worst Pulse Width Slack (WPWS): 1,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 93	Total Number of Endpoints: 93	Total Number of Endpoints: 88

All user specified timing constraints are met.

- Worst Negative Slack (WNS) : Positif (pas de violation de Setup)
- Worst Hold Slack (WHS) : Positif (pas de violation de Hold)

Name	Slack	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock	Exception	Clock Uncertainty
Path 1	0.371	3	30	Instance_LED_g_reg[24]/C	Instance_RGB_b_reg[1]/CE	3.339	0.828	2.511	4.000	clk_out1_clk_wiz_0	clk_out1_clk_wiz_0		0.063
Path 2	0.559	3	30	Instance_LED_g_reg[24]/C	Instance_RGB_b_reg[0]/CE	3.151	0.828	2.323	4.000	clk_out1_clk_wiz_0	clk_out1_clk_wiz_0		0.063
Path 3	0.567	3	30	Instance_LED_g_reg[24]/C	Instance_RGB_b_reg[2]/CE	3.143	0.828	2.315	4.000	clk_out1_clk_wiz_0	clk_out1_clk_wiz_0		0.063
Path 4	0.567	3	30	Instance_LED_g_reg[24]/C	Instance_RGB_b_reg[3]/CE	3.143	0.828	2.315	4.000	clk_out1_clk_wiz_0	clk_out1_clk_wiz_0		0.063
Path 5	0.605	6	2	Instance_LED_g_reg[2]/C	Instance_LED_g_reg[20]/D	3.335	2.086	1.249	4.000	clk_out1_clk_wiz_0	clk_out1_clk_wiz_0		0.063



L'analyse temporelle réalisée par Vivado montre que le Path 1 constitue le chemin critique du design, présentant le Slack (Worst Negative Slack) le plus faible (+0,371 ns) et un délai de propagation total de 3,339 ns (se décomposant en 0,828 ns de délai logique et 2,511 ns de délai d'interconnexion/routage).

Néanmoins, avec un Slack positif de +0,371 ns sur une période d'horloge requise de 4,000 ns (horloge générée clk_out1_clk_wiz_0 à 250 MHz), l'ensemble des contraintes de temps d'établissement est respecté sans aucun défaut temporel (0 failing endpoint).

Le chemin critique extrait (visible sur le schéma technologique) franchit les limites de modules : il s'étend de la bascule de départ `cpt_reg_reg[24]` (au sein du composant `counter_unit_1` du **LED_Driver**) jusqu'à la broche de validation d'horloge (CE) de la bascule d'arrivée `cycle_nb_reg[1]` (dans le module **RGB_Sequencer**).

Physiquement, ce chemin correspond à la propagation du signal de contrôle depuis le bit 24 du compteur dans le module **LED_Driver** jusqu'à l'autorisation d'écriture des registres de séquence du module **RGB_Sequencer**. Le signal traverse 3 niveaux de logique combinatoire (composés exclusivement de deux portes LUT4 et d'une porte LUT6) avant d'atteindre la broche CE du registre de destination.

L'analyse détaillée de ce chemin critique montre que le facteur limitant n'est pas la complexité logique (le délai logique ne représentant que **0,828 ns**, soit 25 % du temps total), mais le délai de routage (**2,511 ns**, soit 75 % du temps total).

Pour affranchir le circuit des limitations temporelles causées par la taille du compteur à 250 MHz, plusieurs leviers d'architecture peuvent être activés. Au niveau du code VHDL, la logique de comptage peut être optimisée en adoptant une **pré-détection à VAL_MAX – 1**. Au lieu d'attendre que le compteur atteigne la valeur finale pour générer la fin de comptage, la valeur **VAL_MAX - 1** est détectée et mémorisée dans une bascule synchrone. L'impulsion sortira exactement au coup d'horloge suivant sans aucun délai combinatoire.

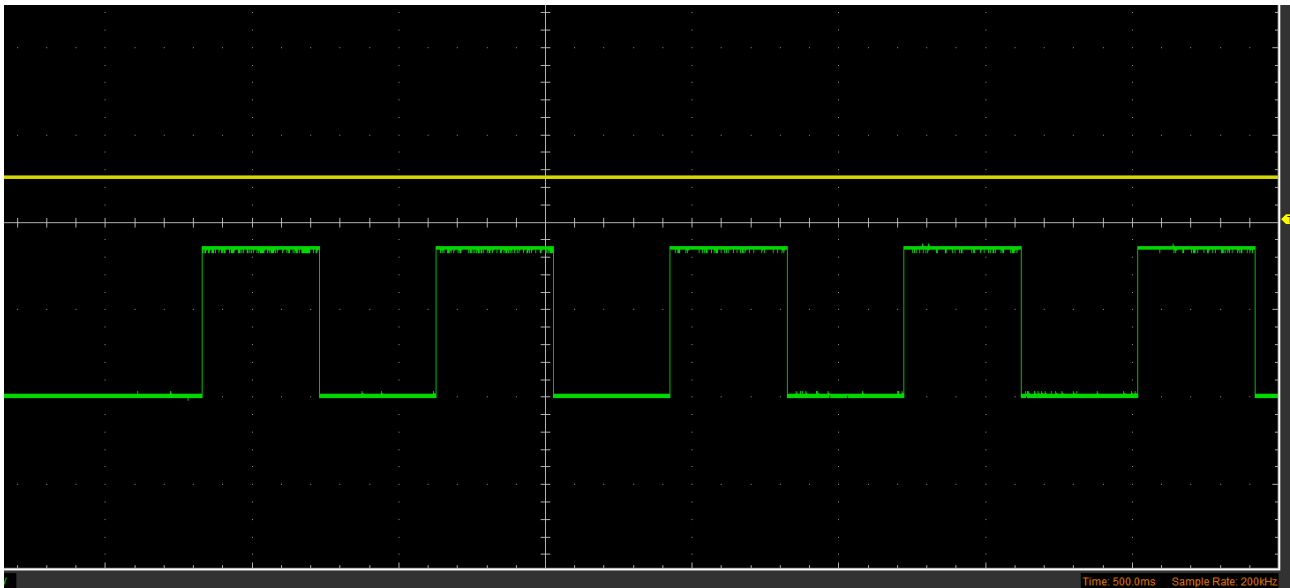
Cependant, une autre solution plutôt efficace sur l'architecture globale consisterait à ajouter un registre de **pipelining** sur le signal '`end_cycle`'. En mémorisant ce signal dans une bascule synchrone, on sépare physiquement le long trajet combinatoire qui traverse les modules **LED_Driver** et **RGB_Sequencer**. Cette modification réduit la longueur des lignes de routage à parcourir en un seul cycle d'horloge (4 ns), élimine les variations de slack observées entre deux compilations.

2.13. Question 13

Une démonstration vidéo est également disponible pour montrer le bon fonctionnement de cette partie de ce TP. (disponible dans le fichier README du dossier TP5 qui dirige vers un lien Google drive).

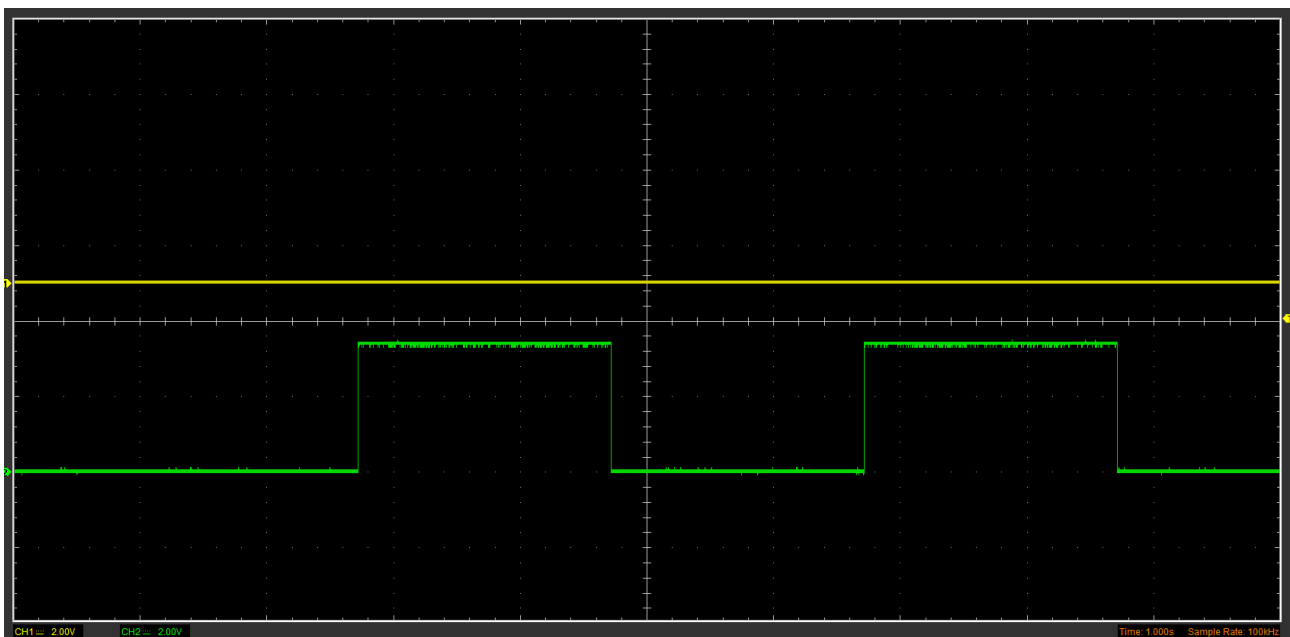
Les différentes fréquences d'horloge sont visualisables à l'oscilloscope, en prenant en compte par exemple les sorties LEDs de couleur Rouge :

LED0 Rouge (250 MHz): Time/DIV de 500 ms



Les cycles ON/OFF se déroulent sur une durée d'un peu moins de 1 seconde chacun.

LED1 Rouge (50 MHz): Time/DIV de 1 s



Les cycles ON/OFF se déroulent sur une durée de 4 s environ chacun.

Les vérifications avec les sondes ILA n'ont pas pu aboutir suite à quelques conflits dans le logiciel provoquant l'absence d'un outil de debug. La démonstration vidéo et la visualisation à l'oscilloscope sont les seules preuves de validation de l'application.